



INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

Course: BVWS103-OWASP Top 10 Vulnerabilities And
Exploitation Techniques

Student Name: Abubakari-Sadique Hamidu

Student ID: 2025/FWSD/11392

Instructor: Aminu Iddris (CCNA, CompTIA, CEH, OSCP, CISSP,
CISM)

Title: Insecure Deserialization

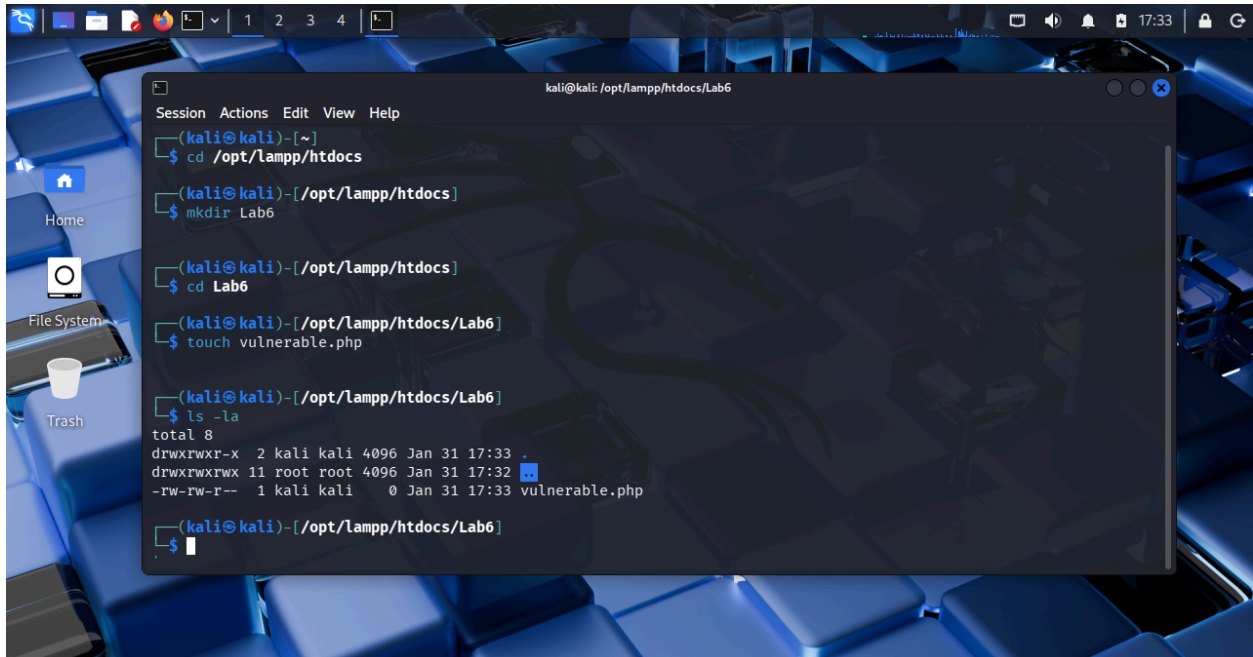
Objective

- Understand insecure deserialization.
- Exploit a vulnerable PHP application.
- Apply proper mitigation to prevent the attack.

Task 1: Creating a Vulnerable Application

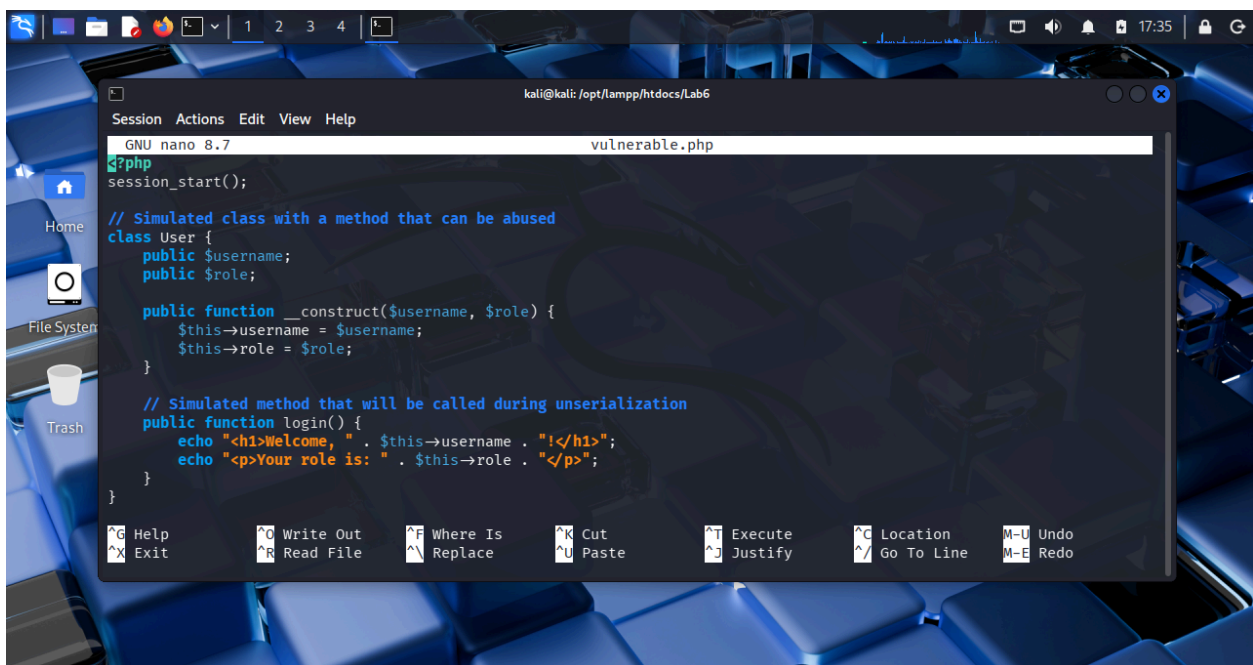
A PHP file named vulnerable.php was created, inside the Lab6 folder created earlier. The application accepts serialized data from user input and directly unserializes it without validation.

This makes the application vulnerable to object injection.



A terminal window on a Kali Linux desktop. The window title is 'kali@kali: /opt/lampp/htdocs/Lab6'. The terminal shows the following commands and output:

```
kali@kali:~$ cd /opt/lampp/htdocs
kali@kali:~/opt/lampp/htdocs$ mkdir Lab6
kali@kali:~/opt/lampp/htdocs$ cd Lab6
kali@kali:~/opt/lampp/htdocs/Lab6$ touch vulnerable.php
kali@kali:~/opt/lampp/htdocs/Lab6$ ls -la
total 8
drwxrwxr-x  2 kali kali 4096 Jan 31 17:33 .
drwxrwxrwx 11 root root 4096 Jan 31 17:32 ..
-rw-rw-r--  1 kali kali   0 Jan 31 17:33 vulnerable.php
kali@kali:~/opt/lampp/htdocs/Lab6$
```



A nano editor window titled 'GNU nano 8.7 vulnerable.php'. The code inside is as follows:

```
?php
session_start();

// Simulated class with a method that can be abused
class User {
    public $username;
    public $role;

    public function __construct($username, $role) {
        $this->username = $username;
        $this->role = $role;
    }

    // Simulated method that will be called during unserialization
    public function login() {
        echo "<h1>Welcome, " . $this->username . "!</h1>";
        echo "<p>Your role is: " . $this->role . "</p>";
    }
}
```

Task 2: Exploiting Insecure Deserialization

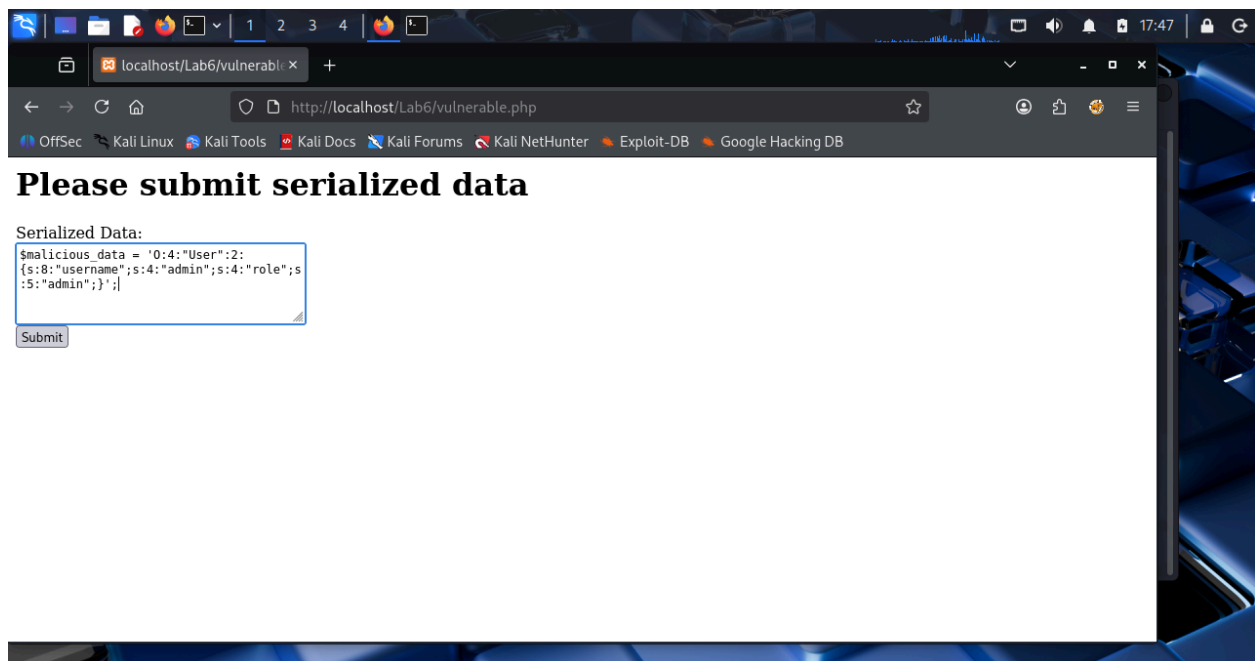
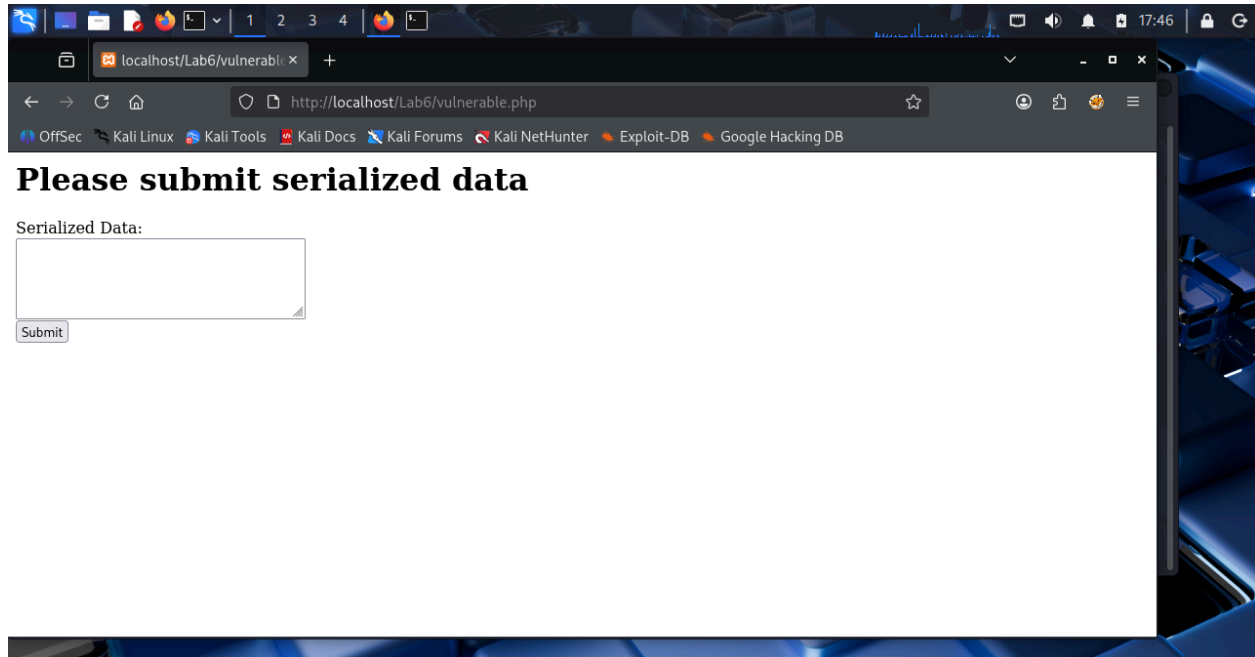
The application was accessed through the browser.

A serialized User object was manually submitted through the form.

Serialized payload used:

```
O:4:"User":2:{s:8:"username";s:5:"admin";s:4:"role";s:5:"admin";}
```

The application trusted the input and created an admin user without authentication.



Observed Issue

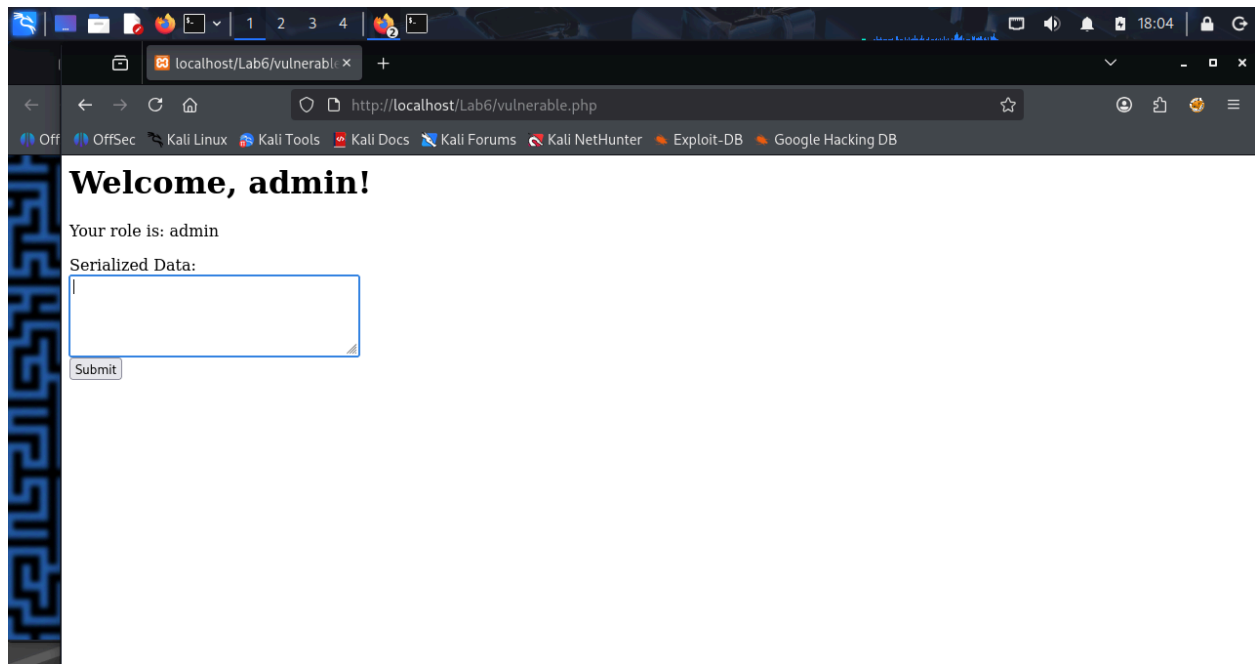
Initially, the application returned a **500 Internal Server Error**.

This occurred due to improper handling of unserialized input.

Error reporting was enabled to reveal and fix the issue.

After correction, the exploit worked successfully.

This shows insecure deserialization can also cause denial of service.



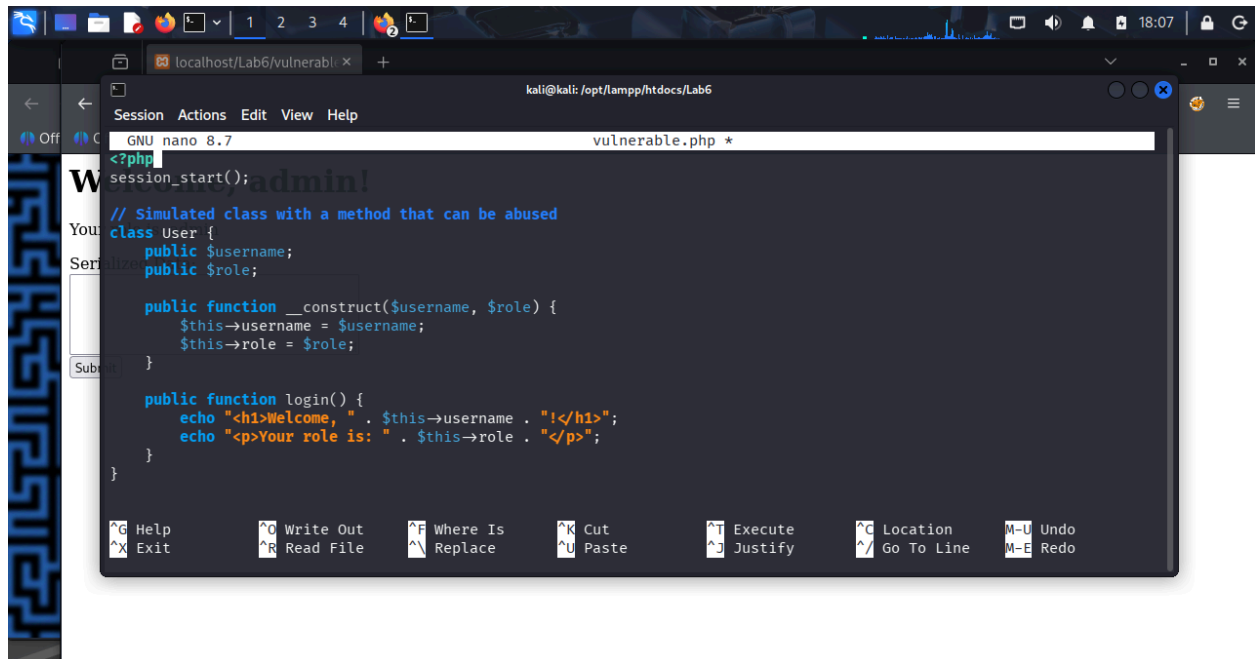
Task 3: Mitigation

The application was refactored to remove unserialize().

Changes made:

- Replaced unserialize() with json_decode()
- Validated input structure before object creation
- Removed trust in user-supplied objects

This prevents object injection.



```
<?php
session_start();
// Simulated class with a method that can be abused
class User {
    public $username;
    public $role;

    public function __construct($username, $role) {
        $this->username = $username;
        $this->role = $role;
    }

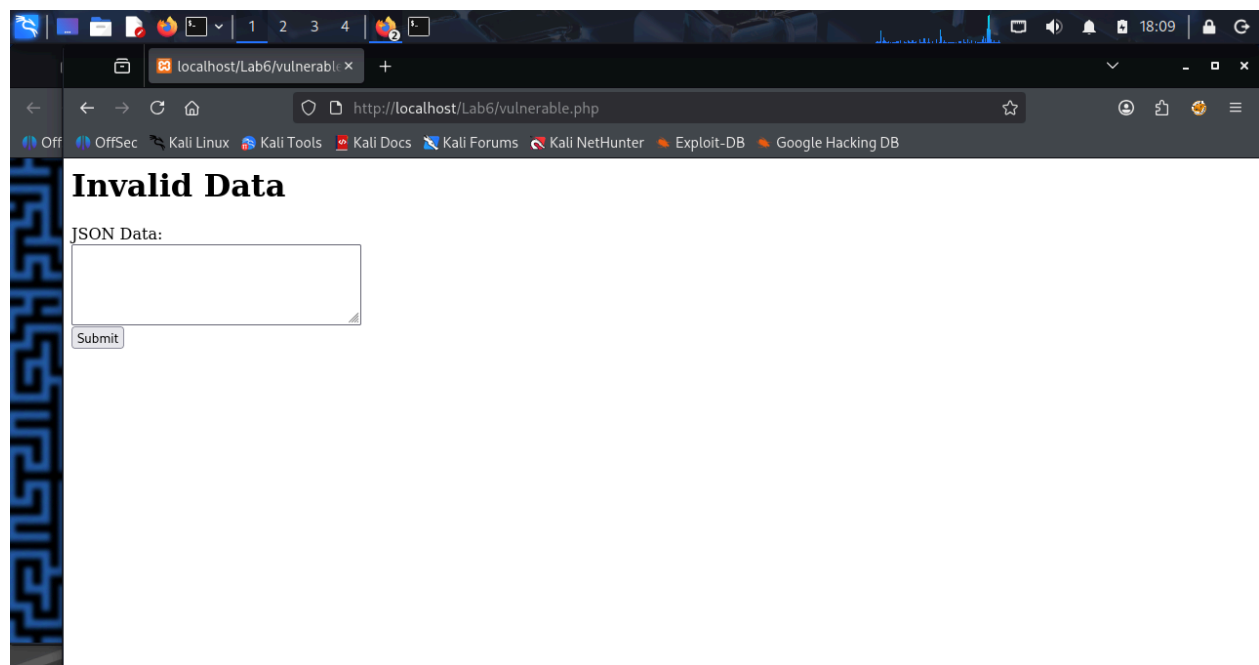
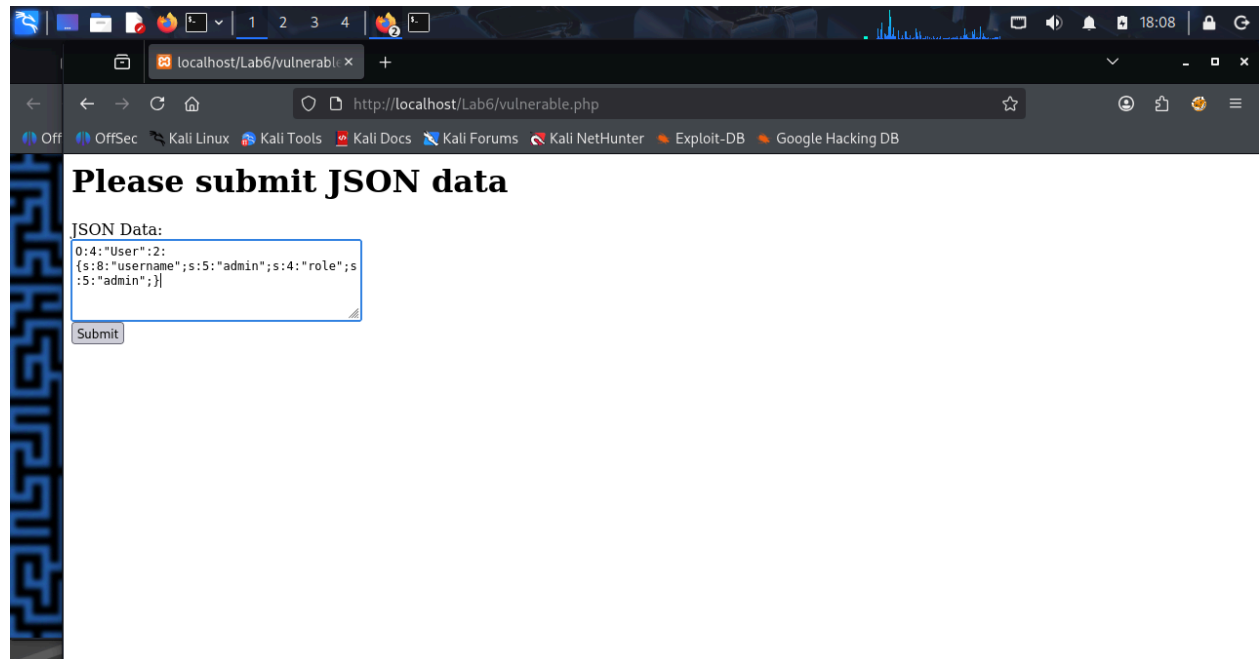
    public function login() {
        echo "<h1>Welcome, " . $this->username . "!</h1>";
        echo "<p>Your role is: " . $this->role . "</p>";
    }
}
```

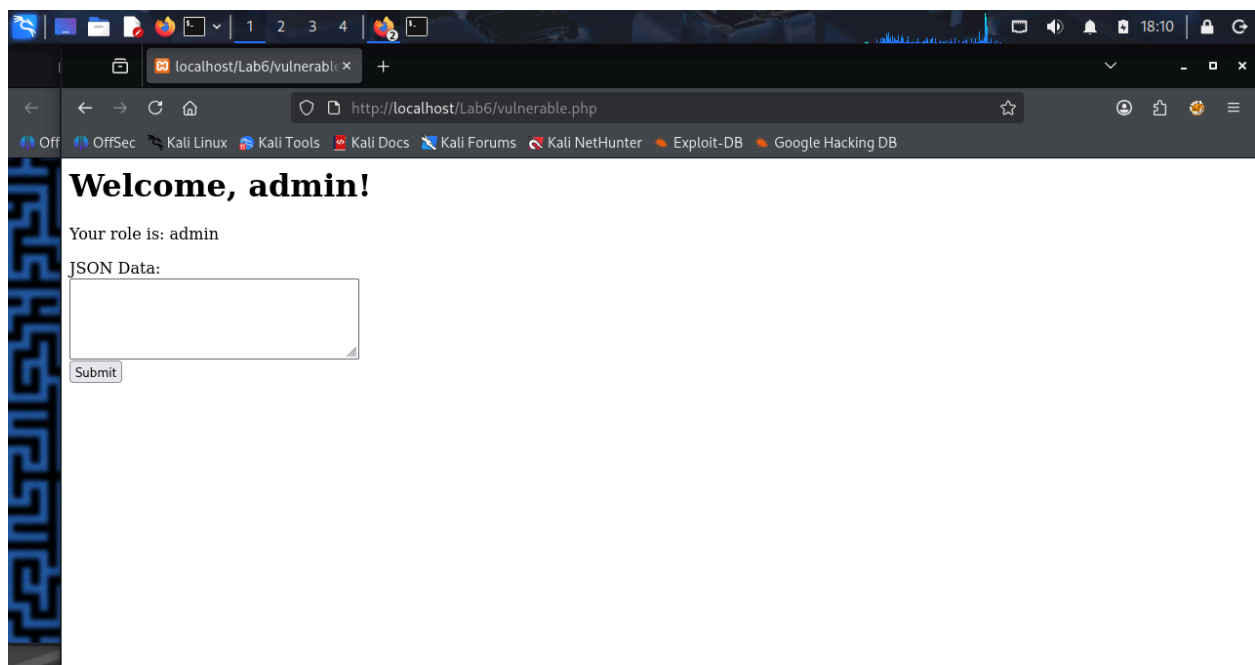
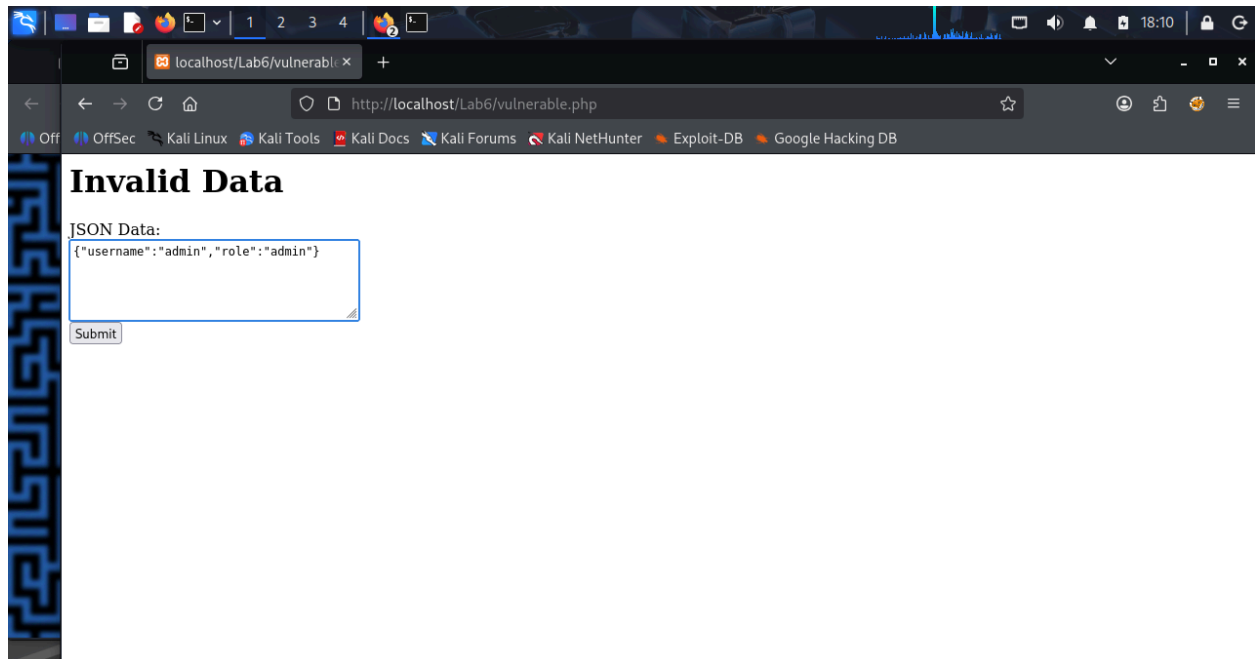
Testing the Fixed Application

- Old serialized payload was rejected.
- Valid JSON data was accepted.

Valid JSON used:

```
{"username": "admin", "role": "admin"}
```





Conclusion

Insecure deserialization allows attackers to inject malicious objects. This can lead to privilege escalation or server crashes.

Using JSON and validating input prevents this vulnerability. Applications must never unserialize untrusted data.